

Polycopié de cours : Systèmes d'Exploitation 5

Ce document propose un polycopié de cours basé sur les cours magistraux de Dominique Poulalhon à l'Université Paris Cité. Il est destiné à servir de support d'apprentissage personnel, mais il peut sans doute être utile à d'autres.

Attention, ce polycopié n'est pas officiel et peut contenir des erreurs. Il est fortement recommandé de se référer aux cours magistraux et aux ressources officielles pour un apprentissage complet et précis.

I Introduction aux Systèmes d'Exploitation

A Système d'exploitation et appels système

Définition : Un **Système d'Exploitation** (OS : Operating System) est une couche logicielle qui fait l'interface entre le matériel (processeur, mémoire, périphériques) et les applications ou l'utilisateur. Il fournit le bon niveau d'abstraction et gère les ressources de façon équitable et sécurisée.

Remarque : Pour garantir la sécurité et l'équité, les processeurs modernes possèdent au moins deux modes de fonctionnement :

- **Mode Utilisateur (User mode) :** Mode bridé où seules les instructions inoffensives sont exécutées.
- **Mode Noyau (Kernel mode) :** Mode privilégié où toutes les instructions et tous les accès matériels sont autorisés.

Vocabulaire : La transition entre le mode utilisateur et le mode noyau s'effectue via un **appel système** (*syscall*).

Remarque : Sur Linux, la commande `strace` permet de tracer les appels système effectués par un programme. La page 2 du manuel fournit l'ensemble des appels système disponibles.

Important :

Un appel système est plus long qu'un appel de fonction classique car il nécessite :

1. Le passage du mode utilisateur au mode noyau (changement de contexte).
2. La sauvegarde et la restauration des registres du processeur.

B Espace d'adressage et états d'un processus

Définition : Un **processus** est l'instance dynamique d'un programme en cours d'exécution. Il possède un espace d'adressage propre divisé en :

- **Code :** Instructions exécutables du programme.
- **Données statiques :** Variables globales et statiques.
- **Tas (Heap) :** Zone d'allocation dynamique (ex. `malloc`).
- **Pile (Stack) :** Zone de stockage des variables locales et contextes d'appel.

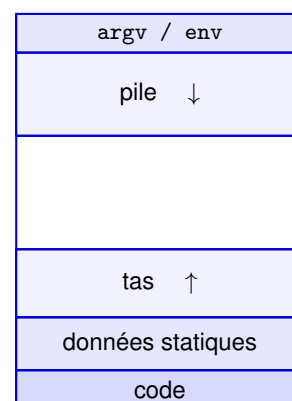


Figure 1: Espace d'adressage mémoire

Remarque : Le processus peut se trouver dans différents états au cours de son cycle de vie. En particulier,

quand une opération est en attente et demande un temps d'attente (ex: lecture sur disque), le processus passe de l'état **ACTIF** à l'état **EN ATTENTE**. Il ne peut revenir à l'état **ACTIF** que lorsqu'un signal ou la fin de l'opération d'entrée/sortie est reçue.

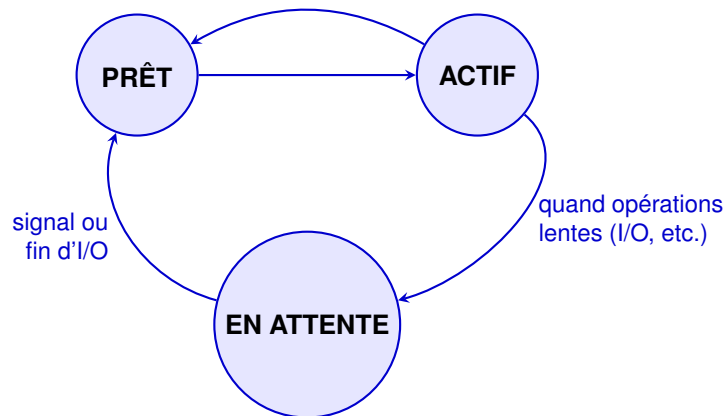


Figure 1: Diagramme des états d'un processus

C Hiérarchie des mémoires

Principe : Hiérarchie de stockage

Pour stocker des données, on doit faire des compromis entre la vitesse d'accès, la capacité et le coût :

1. **Registres** (~ 0.5 ns, très faibles capacités).
2. **Caches** (~ 2 ns).
3. **Mémoire RAM** (~ 10 ns) — volatile.
4. **Stockage secondaire SSD / HDD** (0.1 ms à 10 ms) — non volatile.
5. **Bandes magnétiques** (~ 100 s) — archivage.

II Gestion des Entrées/Sorties et Fichiers

A Descripteurs et tables de fichiers

Définition : Un **descripteur de fichier** (*file descriptor* ou `fd`) est un entier positif renvoyé par un appel système (comme `open`) qui sert d'index à un processus pour accéder à un fichier.

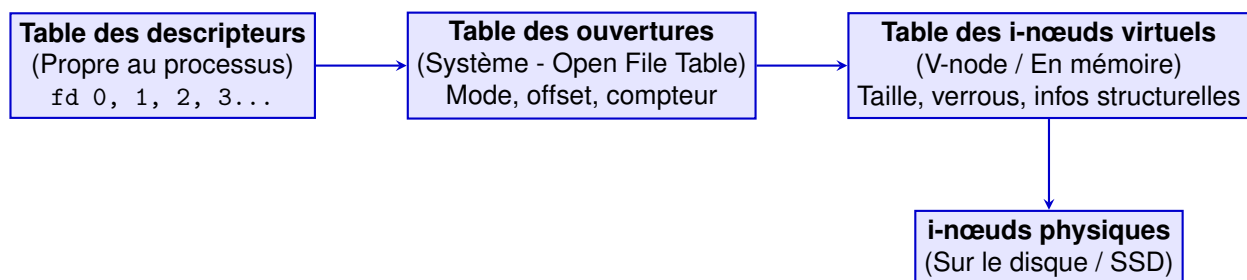


Figure 2: Indirections lors de l'accès à un fichier

Remarque : Tout processus hérite par défaut de 3 descripteurs ouverts :

- 0 : Entrée standard (`STDIN_FILENO`).

- 1 : Sortie standard (STDOUT_FILENO).
- 2 : Sortie d'erreur standard (STDERR_FILENO).

B Appels système fondamentaux pour les I/O

Méthode : Manipuler un fichier via les appels système

1. Ouverture (open) :

```
int open(const char *pathname, int flags, mode_t mode);
```

Les flags obligatoires sont O_RDONLY, O_WRONLY, ou O_RDWR. On peut les combiner avec O_CREAT, O_TRUNC, O_APPEND, O_EXCL via un OR bit-à-bit (|).

2. Lecture (read) :

```
ssize_t read(int fd, void *buf, size_t count);
```

Lit au plus count octets et les place dans buf. Renvoie le nombre d'octets lus, 0 à la fin du fichier, ou -1 en cas d'erreur.

3. Écriture (write) :

```
ssize_t write(int fd, const void *buf, size_t count);
```

Écrit count octets depuis buf dans le fichier.

4. Positionnement (lseek) :

```
off_t lseek(int fd, off_t offset, int whence);
```

Modifie la position de la tête de lecture/écriture selon whence (SEEK_SET, SEEK_CUR, SEEK_END).

5. Fermeture (close) :

```
int close(int fd);
```

Libère le descripteur et met à jour les compteurs dans les tables système.

✗ **Attention** ✗ Certains appels systèmes sont bufferisés par la bibliothèque standard C. Par exemple, `printf` utilise un tampon interne et ne déclenche pas immédiatement un appel système `write`. Pour forcer l'écriture immédiate, on peut utiliser `fflush(stdout)`.

C Surcouche FILE* de la bibliothèque standard

📌 **Remarque** : La bibliothèque standard C (`stdio.h`) propose des fonctions de plus haut niveau manipulant des pointeurs `FILE*`. Un objet `FILE` encapsule un descripteur de fichier et y ajoute un **tampon mémoire (buffer)** pour réduire le nombre d'appels système.

✗ **Attention** ✗ Le buffer d'un flot `FILE*` est vidé (*flushed*) automatiquement dans les cas suivants :

- Quand le tampon est plein.
- Lors d'un appel explicite à `fflush(stream)`.
- À la fermeture avec `fclose(stream)`.
- À la fin normale du processus via `exit` (mais **pas** lors d'un arrêt brutal comme un SIGKILL ou un crash).
- Pour les sorties sur terminal, à chaque saut de ligne ('`\n`').

D Notion d'atomicité

Définition : Une opération est dite **atomique** si elle s'exécute en une seule étape ininterrompue par le système d'exploitation.

💡 **Exemple :** L'ouverture avec `O_CREAT | O_EXCL` garantit que la vérification de l'existence et la création du fichier se font de façon atomique. De même, l'ouverture en mode `O_APPEND` garantit que le repositionnement en fin de fichier et l'écriture s'effectuent atomiquement.

Contents

I	Introduction aux Systèmes d'Exploitation	1
A	Système d'exploitation et appels système	1
B	Espace d'adressage et états d'un processus	1
C	Hierarchie des mémoires	2
II	Gestion des Entrées/Sorties et Fichiers	2
A	Descripteurs et tables de fichiers	2
B	Appels système fondamentaux pour les I/O	3
C	Surcouche FILE* de la bibliothèque standard	3
D	Notion d'atomicité	4

Crédits

Ce cours est inspiré du cours de l'Université Paris Cité réalisés par Dominique Poulalhon. Assurez-vous de citer correctement la source si vous utilisez ce cours dans vos travaux.

Ce cours est mis à disposition selon les termes de la Licence Creative Commons  **Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 4.0 International**. Pour voir une copie de cette licence, visitez <http://creativecommons.org/licenses/by-nc-sa/4.0/>.